

课程笔记

当我们讨论 Agent Harness 时，我们究竟在讨论什么？

五道口纳什 & Codex

2026 年 4 月 20 日



视频作者/频道: MiniMax × *HermesAgent*

发布日期: 2025 年

视频时长: 约 2 小时 6 分钟

视频链接: <https://www.bilibili.com/video/BV1FVdpBTEbB/>

目录

1 Agent 热潮的行业背景——从”养虾”到”驯马”	2
1.1 引言：为什么是现在？	2
1.2 开场介绍与嘉宾背景	2
1.3 ”养虾”现象：OpenClaw 为何引发全民关注	2
1.3.1 现象回顾：从硅谷到上海	2
1.3.2 爆火的三层原因	3
1.4 从”养虾”到”驯马”：用户关注点的转移	3
1.4.1 一个月内的热度更替	3
1.4.2 用户心理的转变：从”出 bug”到”我没训好”	4
1.5 Agent Harness 概念的浮现：为什么是 Harness 而不是 Model？	4
1.5.1 Tommy 的定义：大脑与双手	4
1.5.2 泽音的”同事”比喻	4
1.5.3 阿岛的”高达”与”驯马”比喻	5
1.5.4 为什么”Harness”这个词现在才火？	5
1.6 本章小结	5
2 Hermes Agent 深度解析——开源 Harness 的崛起	7
2.1 Hermes Agent 的定位：不是另一个 Chatbot	7
2.2 开源 Harness 的产品哲学：为什么选择开源？	7
2.2.1 Harness 的本质：约束与自由的平衡	7
2.2.2 为什么 Harness 在这个时刻诞生？	8
2.2.3 开源哲学的深层动机	8
2.3 Self-Evolution：Agent 如何自己学会新技能	8
2.3.1 人类成为瓶颈：Self-Evolution 的原始动机	8
2.3.2 Self-Evolution 与推荐算法的本质区别	9
2.4 Skill 系统与可扩展性设计	10
2.4.1 Multi-Agent：不是限制，而是解放	10
2.4.2 Skill 系统：从经验到可复用能力	10
2.4.3 Harness 作为”高达的机甲”	10
2.5 本章小结	11
3 Agent Harness 的技术内核——约束、协作与实时思考	12
3.1 从”工具”到”伙伴”：用户与 Agent 关系的深层转变	12
3.1.1 ”养虾”心理学：当用户觉得 Agent ”蠢萌”而非”出 bug”	12
3.1.2 Self-Evolution：Agent 如何”越聊越懂你”	12
3.2 Skill 与 COI：让普通人也能扩展 Agent 的能力边界	13
3.2.1 ”蒸馏”的本质：从人类经验到可复用能力	13
3.2.2 Skill + COI vs. MCP：两种扩展范式的对比	13
3.2.3 蒸馏的双向性：谁在蒸馏谁？	14
3.3 In-Live Thinking：Agent 与 Chatbot 的本质分野	14
3.3.1 Chatbot 的结构性局限	14
3.3.2 BrowserCamp：Agent 能力的试金石	15

3.3.3	In-Live Thinking: 边做边想的实时决策机制	15
3.4	模型与应用的共生螺旋	16
3.4.1	”你永远会被模型内化掉”	16
3.4.2	Human in the Loop: 人类在循环中的不可替代性	17
3.5	本章小结	17
4	模型公司与 Agent 的共生关系——竞争、互补与未来形态	18
4.1	模型公司的视角: 为什么 MiniMax 也在做 Agent?	18
4.1.1	一切皆为代码: Agent 的本质洞察	18
4.1.2	中国模型与海外领先者的真实差距	18
4.2	COI 困境: 模型厂商做 Agent 的利益冲突	19
4.2.1	Agent Infra 的两层结构	19
4.2.2	垂直 Agent 的创业路径: 两阶段演化	20
4.3	Agent 会取代模型吗? 还是相辅相成?	20
4.3.1	通用 Agent vs 垂直 Agent: 嘉宾观点碰撞	20
4.3.2	法律: 通用 Agent 走不通的”最后一公里”?	20
4.4	未来产品形态畅想: 从 IDE 到自然交互	21
4.4.1	GPT-4.7: RL 路线的胜利与代价	21
4.4.2	Claude Code 强制实名: 安全还是垄断?	21
4.4.3	Manager Agent 与源代码泄露: 封闭 vs 开放的博弈	22
4.5	本章小结	22
5	开源、安全与人类未来——行业争议与乐观主义收尾	24
5.1	开源 vs 封闭: 两条路径的博弈	24
5.2	AI 安全与对齐: 被忽视的问题	24
5.3	”AI 替代人类”: 嘉宾们的不同观点	25
5.4	乐观主义收尾: 拥抱新版本的”蒸汽机”	26
5.5	本章小结	26
6	总结与延伸	28
6.1	对谈的核心脉络回顾	28
6.2	我们的结构化蒸馏	28
6.3	开放问题与下一步	29

1 Agent 热潮的行业背景——从”养虾”到”驯马”

1.1 引言：为什么是现在？

2025 年春节后的短短一个多月里，人工智能领域发生了一场令人目眩的舆论地震。从 OpenClaw（国内网友戏称”养虾”）的全民爆火，到 Hermes Agent（国内网友戏称”驯马”）的迅速接棒，中国用户对 **Agent** 的认知完成了从”听说过”到”天天用”的跃迁。这场热潮来得急、去得也快，但它在公众心中种下了一个坚定的信念：**AI 不再只是聊天工具，而是可以替你干活的数字同事。**

本章将带领读者回到这场变革的现场，从嘉宾们的第一视角复盘：

- 为什么”养虾”会在中国比海外更火？
- 用户的心理如何从”工具使用者”扭转为”协作者”？
- 为什么”Agent Harness”这个概念会从边缘走向中心？

理解这些背景，是后续深入探讨 Harness 架构、Multi-Agent 协作与技能蒸馏的前提。

1.2 开场介绍与嘉宾背景

本场深度对谈由 MiniMax 首席架构师阿岛、MiniMax Agent 研发工程师泽音，以及远程连线的 Hermes Agent 产品与策略负责人 Tommy 共同参与。三位嘉宾分别代表了 **模型底层**、**Agent 工程实现**与 **开源 Harness 框架**三个关键视角。

MiniMax 与 Hermes Agent

MiniMax 是中国领先的通用人工智能公司，其 M 系列模型（M1、M2.1、M2.5、M2.7）在 Agentic 能力上持续迭代。**Hermes Agent** 则是由开源研究组织 Nous Research 开发的 Agent Harness，在 OpenRouter 上的日调用量从 20 亿 token 飙升至近 3000 亿 token，是 2025 年初增长最快的开源 Agent 项目之一。

阿岛在开场时回顾了自己近三年的 MiniMax 经历，深度参与了 M2 系列模型、海螺视频模型等研发。他特别提到，早在 MiniMax IPO 上市当天，团队就与 OpenClaw 创始人 Peter 建立了联系——那一天也恰逢 Anthropic 封禁了 OpenClaw 对其 API 的订阅使用。泽音则负责 MiniMax 官网 Agent、MaxCloud 以及新近上线的 MiniMax Armies 的开发，对 Agent 的工程落地有直接的体感。

Tommy 作为 Hermes Agent 的负责人，代表 Nous Research 首次与中国开发者社区直接对话。Nous Research 起源于 2022 年的一个 Discord 频道，由热爱开源和 AI 模型的研究者自发聚集而成，先后产出过 Llama 后训练模型（即早期的 Hermes 系列）、YaRN 长上下文算法以及分布式训练优化器。Tommy 强调，Hermes Agent 的诞生非常”功利”——联合创始人 Technium 只是想给自己造一个能管理日常工作和后训练流程的助手，**没有 benchmark、没有 KPI，只有一个目标：让它为自己干活。**

1.3 ”养虾”现象：OpenClaw 为何引发全民关注

1.3.1 现象回顾：从硅谷到上海

2025 年 1 月，OpenClaw 在海外开始走红，MiniMax 的 API 甚至一度”售罄”。令人意外的是，这股热潮跨太平洋抵达中国后，**烧得比原产地更旺**。阿岛回忆道：

”在春节期间以及春节之后，就好像身边所有人都开始在讨论龙虾（OpenClaw），甚至我们看到各地都开始出现头戴纱帽的这样的一个热潮，还有腾讯有这样的线下的装机活动。”

OpenClaw 与”养虾”

OpenClaw 是一款基于即时通讯（IM）界面运行的通用 Agent，用户可以通过类似聊天的方式让它执行代码、检索信息、管理文件等任务。国内网友因其 Logo 形似虾钳，戏称其为”养虾”——用户像养宠物一样”调教”自己的 Agent，甚至为其设定性格、记忆和日常作息。

1.3.2 爆火的三层原因

嘉宾们从三个层面剖析了 OpenClaw 在中国爆火的深层逻辑：

第一层：体验落差与”捅破窗户纸”。阿岛指出，海外用户此前已经接触过 Claude Code、Codex、Coworker 等较为成熟的 Agent 产品，体验是渐进式的；而国内用户此前缺乏好用的 Agent，OpenClaw 通过 IM 这一最熟悉的交互方式，让中国用户完成了**从 0 到 1 的跨越**，”一下被捅破了那层窗户纸”。

第二层：国产模型能力的成熟。泽音补充道，国内模型在更早阶段缺乏足够的 Agentic（智能体）能力，而随着 MiniMax M2.5、M2.7 等模型的发布，**国产模型已经具备了支撑 Agent 运行的基础能力**。模型与产品的结合，让”养虾”从玩具变成了工具。

第三层：字节跳动的全民教育。阿岛特别提到了字节跳动在春晚期间对”豆包”的 All-in 投入。尽管字节内部起初也不确定能否承接住如此巨大的流量，但他们坚定执行，最终让”豆包”的渗透率达到了惊人的水平。**OpenClaw 与豆包，两件事结合在一起，让所有人对 AI 的认知真正进入了 Agent 时代。**

常见误解：Agent

很多初次接触 Agent 的用户会将其与 Chatbot 混为一谈。关键区别在于：Chatbot 的核心是”当下给你一个回答”，即使加入了 Reasoning，它也不能与环境做大量互动；而 Agent 的核心能力是**在与环境互动的过程中不断推理、纠正执行路径，最终交付成果**。OpenClaw 的价值不在于”能聊天”，而在于”能干活”。

1.4 从”养虾”到”驯马”：用户关注点的转移

1.4.1 一个月内的热度更替

OpenClaw 的爆火到 Hermes Agent（”驯马”）的接棒，大约只花了一个月。泽音将这一过程概括为**双向奔赴**：

”在这一个月之内，大家基本上都已经体验过了一下 OpenClaw……但其实 OpenClaw 里面，大家可能也会遇到一些特别的问题……它每天四点钟会刷新一次记忆，这时候你可能比较好的和他交流过以后，过了一天他就会说’诶我们昨天聊了什么，我忘记掉了’。”

OpenClaw 的痛点恰好成为 Hermes Agent 的突破口。Hermes Agent 在记忆方面做了多层次的设计，弥补了 OpenClaw 的不足。更重要的是，用户在”养虾”过程中已经享受到了 AI 提效的快乐，**对让 AI 介入自己生活的接受度显著提高**——这为更强大的 Agent 出现铺平了道路。

1.4.2 用户心理的转变：从”出 bug”到”我没训好”

这场对话中最具洞察力的观察之一，是用户与 AI 关系发生的根本性扭转。主持人提到：

”当这个养小龙虾的时候，他挂机了、他宕机了，或者他一个任务完不成，我们只会觉得他挺蠢萌的，甚至有时候会觉得’哎是我好像没把它训好’，我要去网上学一下别人再怎么训他的虾。”

这种心理转变意味着：

- **从工具使用者到协作者**：用户不再认为”AI 出错 = 产品 bug”，而是反思”我是不是没辅佐好”。
- **从人类中心到 AI 中心**：阿岛将其类比为**蒸汽机和电力**的出现——”人类围绕它重新去构建你的生产方式、你的工作流”。
- **信任基点的提升**：当人对 AI 的信任发生变化后，会交给它更复杂的任务，人类社会的工作方式也会随之改变。

核心转折：信任是 Agent 普及的基石

阿岛指出：”AI 能力的基点，当人对它的信任发生了变化之后，会交给它的任务不同，而且人类社会以及人类的工作方式会围绕它改变，而不是尝试去改变它。”这一观点来自 OpenAI 负责 Harness 的研究者以及 Claude 之父 Boris 的公开分享——**不要想模型做不到什么，要想模型能够做到什么，以及你应该如何让它做到。**

1.5 Agent Harness 概念的浮现：为什么是 Harness 而不是 Model？

1.5.1 Tommy 的定义：大脑与双手

当 Tommy 被问及”什么是 Hermes Agent”时，他给出了一个精妙的比喻：

”Hermes Agent is put simply an open source agent harness. Harness is just the framework that an LLM uses to operate out in the world. Like if the model is the brain, then the agent harness is the hands.”

Agent Harness 的定义

Agent Harness 是大型语言模型（LLM）在现实世界中运作所依赖的框架。它负责：

- 工具编排（Tool Orchestration）与编排循环（Orchestration Loop）
- 状态管理（State Management）
- 错误处理（Error Handling）
- 让模型最大限度地发挥其对用户的实用价值

如果模型是大脑，Harness 就是双手——**没有 Harness，再聪明的模型也无法触碰现实世界。**

1.5.2 泽音的”同事”比喻

泽音从工程视角进一步阐释了 Harness 的本质：

”它其实就是你去约束 Agent，但是又给他一定自由，同时的话他就能够完整的交付给你一些成果的这样一套玩法……你就假设一下你有一个同事，然后他全都听你的，但是你得和他先约定好他能干什么、你不能干什么。”

泽音将 Harness 的核心要素归纳为：

1. **工具与环境**：给 Agent 配置模型、工具、权限，如同给同事配备笔记本、电话和邮箱账号。
2. **约束与自由度**：约定边界，有些事情可以放任去做，有些事情必须审批。
3. **Multi-Agent 协作**：引入多位”同事”互相监督、交叉验证（Cross-check），形成制约与对抗，产出单个 Agent 无法完成的高质量结果。

1.5.3 阿岛的”高达”与”驯马”比喻

阿岛则用两个更为生动的比喻，揭示了 Harness 出现的必然性：

”我那个年代的东西叫高达 W……他的特点就是说他引擎非常的强大，我们其实需要去造一个高达或者那样的一个机甲，然后怎么让这个引擎的能力最大的发挥出来……所以我认为 Harness 就是那样一个东西。”

”你有一匹很厉害的烈马，但怎么样让他最能够发挥出来，就是构造 Harness——对，那样的一个过程。”

这两个比喻共同指向一个核心论点：**模型的智能（引擎/烈马）需要 Harness（机甲/马鞍）才能在实际工作中发挥最大价值。**

1.5.4 为什么”Harness”这个词现在才火？

阿岛指出，Harness 的实践早在 OpenAI 官方文章发布之前就已存在。大约 2024 年 9 月，他自己的工作流中已经同时并发五六个本地 Agent、云端十几个 Agent 在 GitHub 上尝试不同分支——**人类反而成了瓶颈**：

”自己人类成为瓶颈……因为我们需要不断的在这些上下文中切换，然后给他们输入。所以这是我们想的一个东西：怎么样让它能自己更自动化。”

OpenAI 的那篇文章，本质上只是**捅破了那层窗户纸**——让行业形成了共识：“我们用什么词来形容我们在做的一切？”这个词就是 Harness。

常见误解：Harness 是新鲜事物

Harness 并非 2025 年才出现的新概念。Tommy 透露，Hermes Agent 的代码仓库已经存在约一年，Technium 个人使用它作为助手的时间更早。OpenAI 的文章只是将业已存在的实践**命名并概念化**，让更多人意识到：模型之上，还需要一层”驾驭”的框架。

1.6 本章小结

本章从 2025 年春节后的 Agent 热潮出发，梳理了三个关键脉络：

1. **社会背景**：OpenClaw 的爆火是国产模型能力成熟、字节跳动全民 AI 教育、以及 IM 低门槛交互三者叠加的结果。它让中国用户首次大规模体验到”AI 能干活”。

2. **用户心理**：从”养虾”到”驯马”，用户与 AI 的关系发生了根本转变——从”工具使用者”变为”协作者”，从”出 bug”变为”我没训好”。**信任的提升**是 Agent 从玩具走向生产工具的心理基石。
3. **概念浮现**：Agent Harness 的出现，标志着行业认知从”拼模型智商”转向”拼模型利用率”。无论是 Tommy 的”大脑与双手”、泽音的”同事协作”，还是阿岛的”高达机甲”与”驯马”，都指向同一个结论：**模型是引擎，Harness 是让其发挥最大效能的框架**。

Takeaway

理解 Agent Harness 的起点，不是技术细节，而是**人如何与 AI 建立信任**。当用户愿意把任务交给 Agent、愿意让 AI 接触自己的数据、甚至愿意为 Agent 的”失误”反思自己的”训练方式”时，Harness 才有了存在的土壤。2025 年的春节，正是这片土壤被开垦的时刻。

下一章，我们将深入 Harness 的技术架构，探讨工具编排、记忆系统与 Multi-Agent 协作的工程实现。

2 Hermes Agent 深度解析——开源 Harness 的崛起

2.1 Hermes Agent 的定位：不是另一个 Chatbot

在深入讨论 Harness 的技术细节之前，有必要先厘清一个根本性的认知问题：**Hermes Agent 到底是什么？**它与我们日常使用的 Chatbot（如 ChatGPT、Claude 的网页对话界面）有何本质区别？

从嘉宾的分享中可以提炼出一个核心洞察：Hermes Agent 的用户群体表现出极强的“模型好奇心”（model curious）。平台上每天有 10 到 12 个不同模型产生非常可观的 token 使用量，其中 MiniMax M2.7 更是达到了单日 250 亿 token 的调用规模。这揭示了一个重要现象——Hermes Agent 的用户不是在寻找一个“更好的聊天机器人”，而是在寻找一个**能够让他们自由实验、对比、组合不同模型能力的开放工作台**。

核心区分：Chatbot vs. Agent Harness

- **Chatbot**：以对话为核心交互形态，用户与单一模型进行问答式交互。
- **Agent Harness**：以任务交付为核心目标，提供模型选择、工具配置、多 Agent 协作、自我进化等完整基础设施，让 Agent 能够自主完成复杂 workflow 并交付成果。

嘉宾特别提到，Hermes Agent 与 MiniMax 的合作是“超级有机地”（super organically）开始的——从 Twitter 上的一条私信，到线下活动，再到深度技术合作。这种合作关系的演进本身也说明了 Hermes Agent 的定位：它不是一个封闭的对话产品，而是一个**开放的 Agent 基础设施**，能够与各种底层模型深度集成，共同推进 Agentic 模型的训练与优化。

背景知识：Open Source 模型的崛起

嘉宾在对话中明确指出：“China clearly holds the lead in open source models right now.”（中国目前在开源模型领域明显处于领先地位。）虽然 Opus 等闭源模型仍位于整体能力的顶端，但开源与闭源之间的差距从未像今天这样小，而且这一差距还在继续缩小。这为 Hermes Agent 这样的开源 Harness 提供了坚实的底层基础——用户可以在 Harness 中自由切换包括 MiniMax M2.7 在内的多种开源模型，找到最适合自己场景的解决方案。

2.2 开源 Harness 的产品哲学：为什么选择开源？

2.2.1 Harness 的本质：约束与自由的平衡

在 00:34:00 左右，嘉宾用一段极其生动的比喻阐释了 Harness 的核心概念：

”Harness 这东西，看起来就是一个英文单词，而且很难念。但是在我看来的话，它其实就是一个你去约束 Agent，但是又给他一定自由，同时的话他就能够完整地交付给你一些成果的这样一套玩法。”

这个比喻可以进一步展开：想象你有一位新同事，你首先需要和他约定好——他能做什么、不能做什么。然后你给他配置工具：一台笔记本电脑、一个电话、一个邮箱账号。当他开始工作时，你并不放心他一个人干活，于是你引入多位同事，让他们之间互相监督、形成制约。最终，他们能够协作完成单个同事完全无法产出的更高质量成果，而你在此期间几乎不需要额外介入。

Harness 的三要素

1. **工具与环境**：给 Agent 配置模型、工具、API 访问权限等”工作能力”。
2. **约束与目标**：设定 Agent 能做什么、不能做什么的边界，以及互相对抗性的审查目标。
3. **多 Agent 协作**：通过多个 Agent 的组合与互相监督，突破单 Agent 的能力上限。

2.2.2 为什么 Harness 在这个时刻诞生？

嘉宾提出了 Harness 出现的三个”物理条件”：

1. **模型变得更聪明了**：当前的大模型已经具备了基本的 Agentic 能力，能够理解复杂指令、调用工具、进行多步推理。
2. **人们愿意放下怀疑**：随着 Agent 在实际场景中展现出价值，用户愿意将更多敏感操作（如操作邮箱、服务部署、财务数据访问）授权给 Agent。
3. **从 Agent 中获得真实收益**：当用户切实体验到 Agent 带来的效率提升后，对更复杂、更自主的 Agent 系统的需求自然涌现。

常见误解：Harness = 简单的工具调用框架

很多人误以为 Harness 只是一个让模型调用外部工具的”胶水层”。但实际上，Harness 的核心价值在于 **orchestration（编排）**——它不仅要让 Agent 能做事，还要让多个 Agent 在约束下高效协作、互相审查、最终交付可靠成果。工具调用只是最底层的能力，Harness 关注的是如何让这些能力在复杂 workflows 中发挥最大价值。

2.2.3 开源哲学的深层动机

嘉宾在回应关于 Hermes Agent 与某开源项目相似性的质疑时，给出了一个强有力的回答：

”We’re deeply ideologically driven team right, and have no interest in really doing anything other than championing open source. We certainly aren’t copying repos and trying to claim them for ourselves.”

这段话揭示了 Hermes Agent 团队核心理念：他们是一个”深度意识形态驱动”的团队，唯一的目标就是推动开源。这种开源不仅仅是代码的公开，更是一种**对 AI 基础设施民主化的信念**——让每个人都能拥有 7×24 小时可用的智能 Agent，让”所有人都能从事创造性工作，而重复性工作留给 Agent”。

2.3 Self-Evolution: Agent 如何自己学会新技能

2.3.1 人类成为瓶颈：Self-Evolution 的原始动机

在讨论 Self-Evolution（自我进化）之前，嘉宾先提出了一个反直觉的观察：**当 Agent 能力足够强时，人类反而成为了瓶颈。**

”我成为一个极大的瓶颈……自己人类成为瓶颈。其实那时候已经不是 Agent 成为瓶颈了。因为我需要不断地在这些上下文中切换，然后给他们输入。”

嘉宾描述了一个真实的场景：他在本地同时并发五六个 Agent，在云端还有十个 Agent 在 GitHub 上尝试不同的分支和想法。此时，他作为人类管理者，需要在不同 Agent 的上下文之间不断切换、输入指令、审查结果——这种人工介入成为了整个系统效率的最大瓶颈。

Self-Evolution 的核心机制

Self-Evolution 的本质是建立一个**反馈闭环**，让 Agent 能够从真实环境中获得反馈，并据此不断沉淀和优化自己的能力：

1. **获取真实反馈**：Agent 不再依赖人类的逐条确认，而是通过 API 测试、代码部署验证、实际运行结果等方式获得客观反馈。
2. **能力沉淀**：成功的经验被沉淀为 Skill、COI (Chain of Instructions) 或 Hooks，成为 Agent 的”肌肉记忆”。
3. **持续优化**：通过反复执行和验证，Agent 在特定领域的能力不断精进，减少对人类的依赖。

2.3.2 Self-Evolution 与推荐算法的本质区别

主持人提出了一个精妙的类比：”我们今天刷抖音也是越刷算法越准，但是我们好像也并没有说抖音会自我进化。”

嘉宾的回答从两个维度澄清了这一区别：

第一，效率维度。在高复杂度、高密度的任务上，如果要 scale up（规模化），就必须让 AI 占据主导地位，人类只是”驾驭”（harness）而非事无巨细地参与。模型训练就是一个典型场景——MiniMax M2.7 的 RL（强化学习）pipeline 中，70%-80% 的工作已经由模型和 Agent 自主完成。人类工程师的角色从”执行具体实验步骤”转变为”基于汇总结果进行方向性判断和品味选择”。

第二，可靠性维度。Self-Evolution 不仅关乎效率，更关乎 Agent 能否解决足够复杂、足够长程的问题。只有当 Agent 能够可靠地长时间自主运行时，自我进化才有意义——”在过去他跑不了那么久的话，也无从谈起自我进化”。

常见误解：Self-Evolution = 无监督的任意生长

Self-Evolution 绝非让 Agent 无约束地”野蛮生长”。嘉宾强调，人类的**判断和 taste（品味）**仍然至关重要。在 MiniMax 的训练流程中，人类工程师负责：

- 审视实验汇总结果与预期方向的偏差；
- 基于品味和创造力指引优化方向；
- 与 Agent 进行讨论（discuss），而非直接执行细节。

Self-Evolution 是”人类驾驭下的进化”，而非”脱离人类控制的进化”。

2.4 Skill 系统与可扩展性设计

2.4.1 Multi-Agent：不是限制，而是解放

在深入 Skill 系统之前，有必要先理解为什么 Hermes Agent 选择多 Agent 架构。这涉及两个核心论证：

论证一：信息交换效率。

嘉宾提出了一个思想实验：当你与单个 Agent 交互时，你输入的信息量很少（”批准”、”接着干”、”加油”），而 Agent 输出的信息量很大（一篇”小作文”）。这种**信息不对称**导致整体效率低下。但如果让两个模型之间以极高密度交换信息，它们的协作效率将远超人与单个 Agent 的交互。

论证二：上下文衰减与纠错能力。

嘉宾引用了一个已被论证的现象：当 Agent 的上下文空间使用超过 50% 后，其智能水平会呈指数级下降。这是单 Agent 的**结构性上限**。更重要的是，Agent 在训练数据中看到的大部分是线性执行轨迹，较少看到反复验证和思考的模式——这与人类”钻牛角尖”的行为类似。

”如果你设置两个 Agent 的话，它可以没有之前的心智负担，换一个角度重新思考……往往能达到一个更高的质量。”

另一位嘉宾补充了 Cross-Check（交叉验证）的实践：在出方案时让两个 Agent 互相审查，当某个 Agent 偏离正确路径时，另一个 Agent 能够及时纠正——”就像你在钻牛角尖的时候，旁边有一个人过来泼一盆冷水，让你清醒一下”。

2.4.2 Skill 系统：从经验到可复用能力

Skill 系统是 Hermes Agent 可扩展性设计的核心。在 Self-Evolution 的反馈闭环中，Agent 将成功经验沉淀为可复用的 Skill。这些 Skill 可以是：

- **Skill**：完成特定任务的完整能力模块；
- **COI (Chain of Instructions)**：指令链，描述完成某类任务的标准化步骤；
- **Hooks**：触发器机制，在特定条件下自动提醒或干预 Agent 的行为。

背景知识：Skill 系统的设计模式

Skill 系统的设计借鉴了软件工程中的**插件架构**（Plugin Architecture）和**能力即服务**（Capability-as-a-Service）思想：

1. **模块化**：每个 Skill 是独立的能力单元，可以单独开发、测试、部署。
2. **可组合**：多个 Skill 可以组合成更复杂的工作流，类似于函数式编程中的函数组合。
3. **可进化**：Skill 本身可以通过 Self-Evolution 机制不断优化，形成”能力的新陈代谢”。

这种设计使得 Hermes Agent 不会随着功能增加而变得臃肿——新能力以 Skill 的形式动态加载，而非硬编码到核心系统中。

2.4.3 Harness 作为”高达的机甲”

嘉宾用一个极具画面感的比喻总结了 Harness 与模型之间的关系：

”你有……高达 W，然后他的特点就是他的引擎非常的强大。我们其实需要去造一个高达或者那样的一个机甲，然后怎么让这个引擎的能力最大地发挥出来。所以我认为 Harness 就是那样一个东西。他们为什么把它叫 Harness？就是你有一匹很厉害的烈马，但怎么样让他最能够发挥出来，就是构造 Harness。”

这个比喻精准地捕捉了 Harness 的本质：

- **引擎/烈马**：底层大模型的智能能力。
- **机甲/Harness**：约束、引导、放大模型能力的系统性框架。
- **目标**：不是让烈马随意奔跑，而是让它在特定方向上发挥最大价值。

2.5 本章小结

本章围绕 Hermes Agent 这一开源 Harness 展开了四个层面的深入解析：

第一，定位层面。 Hermes Agent 不是另一个 Chatbot，而是一个让用户能够自由实验、组合不同模型能力的开放 Agent 工作台。其用户群体的”模型好奇心”和高达 250 亿 token/日的多模型调用量，充分证明了这一定位的成功。

第二，哲学层面。 Harness 的本质是在**约束与自由之间寻找平衡**——给 Agent 工具、环境、自由度，同时通过目标设定和多 Agent 互相监督形成约束。开源哲学不仅是代码开放，更是对 AI 基础设施民主化的深层信念。

第三，机制层面。 Self-Evolution 的核心是建立**真实反馈闭环**，让 Agent 从实际执行中获得反馈、沉淀 Skill、持续优化。但这绝非无监督的任意生长——人类的判断和品味仍然是方向性的驾驭者。

第四，架构层面。 Multi-Agent 架构不是对单模型能力的限制，而是对**上下文衰减和纠错能力**两大结构性问题的系统性解决。Skill 系统则以模块化、可组合、可进化的设计，确保了 Harness 的长期可扩展性。

核心 Takeaway

1. **Harness 是模型的放大器，而非替代者。**它的目标不是取代底层模型，而是让模型的智能在复杂工作流中发挥最大价值。
2. **开源 Harness 的优势在于生态组合。**用户可以自由选择模型、自由组合能力、自由扩展 Skill——这种开放性是封闭生态无法比拟的。
3. **Self-Evolution 的边界是人类的品味。**在 70%-80% 的执行层面让 Agent 自主完成，在 20%-30% 的方向层面保留人类的判断和创造力——这是当前最高效的协作模式。
4. **多 Agent 不是过度设计，而是结构性必需。**上下文衰减、信息交换效率、纠错能力三大因素决定了单 Agent 存在不可突破的上限。

3 Agent Harness 的技术内核——约束、协作与实时思考

本章将讨论推向 Agent Harness 的技术本质。经过前两章对行业背景和开源产品理念的介绍，读者自然会问：**Harness 在技术上究竟解决了什么问题？**嘉宾们围绕用户与 Agent 的关系演化、Skill 系统的设计范式、In-Live Thinking 的技术必要性，以及模型与应用的共生机制，展开了一场从技术哲学到工程实现的深入对话。

3.1 从”工具”到”伙伴”：用户与 Agent 关系的深层转变

3.1.1 ”养虾”心理学：当用户觉得 Agent ”蠢萌”而非”出 bug”

在讨论 Harness 的技术架构之前，有必要先理解一个前置条件：**用户与 Agent 的心理关系正在发生根本性的扭转。**

主持人分享了一个极具洞察力的观察：

”当这个养小龙虾的时候，他挂机了、他宕机了，或者他一个任务完不成，我们只会觉得他挺蠢萌的，甚至有时候会觉得‘哎是我好像没把它训好’，我要去网上学一下别人再怎么训他的虾。”

这种心理转变意味着三层技术含义：

1. **容错容忍度显著提升。**用户不再因为 Agent 的一次失败就放弃使用，而是愿意反复尝试、调整提示词、优化工作流——这为 Harness 的迭代优化提供了宝贵的”用户耐心窗口”。
2. **从”指令-响应”到”共同成长”。**用户开始将 Agent 视为一个需要”培养”的伙伴，而非一次性的工具。这种关系模式要求 Harness 具备**记忆沉淀**和**个性化适配**的能力。
3. **情感连接成为黏性来源。**泽音提到 MiniMax 内部同事”养虾”的场景：”大家在聊的时候，可能有一天这个小龙虾出 bug，他会心里有些小落寞。”这种情感投入使得用户对 Agent 的容忍度和期待值同步提高，也为 Self-Evolution 机制提供了心理基础。

核心洞察：关系扭转是 Agent 普及的心理基础设施

阿岛将这一转变类比为**蒸汽机和电力**的出现：”人类围绕它重新去构建你的生产方式、你的工作流。”当人对 AI 的信任发生变化后，会交给它更复杂的任务，人类社会的工作方式也会随之改变。Harness 的设计必须服务于这种新型关系，而非旧式的”工具调用”范式。

3.1.2 Self-Evolution：Agent 如何”越聊越懂你”

基于上述关系转变，嘉宾进一步引出了 Self-Evolution 在用户侧的价值。阿岛描述了一个理想的 Agent 使用体验：

”我能够在和你对话中越来越了解你，同时的话呢我也能够去给你创造一些小惊喜。就是我不和你对话的时候，或者我就是在一些工作的复盘里，我就能够领悟出一些更加高层次的东西。”

这段话揭示了一个重要的技术方向：**Agent 的进化不应仅发生在与人类交互的实时对话中，还应发生在离线的自我复盘与反思中。**这种”异步进化”机制对 Harness 提出了新的架构要求：

- **持久化记忆层**：Agent 需要维护一个跨会话的长期记忆，记录用户的偏好、习惯和过往交互模式。
- **离线反思能力**：在没有用户输入时，Agent 能够主动回顾历史任务，提炼规律，优化自身的决策策略。
- **惊喜生成机制**：基于对用户需求的深度理解，Agent 能够在恰当的时机主动提供用户尚未明确提出但潜在需要的服务。

3.2 Skill 与 COI：让普通人也能扩展 Agent 的能力边界

3.2.1 ”蒸馏”的本质：从人类经验到可复用能力

在 00:53:00 左右，对话围绕”蒸馏”概念展开了一场精彩的讨论。主持人首先提出了真人流/真同事类 Agent 的现象，阿岛则将其归结为”人们希望和更聪明的人聊天”这一自然诉求。在此基础上，他澄清了”蒸馏”一词的技术含义：

”他实际上就是一个传递信息的过程。我把乔布斯信息、我把巴菲特这种股神的信息，以及我把我同事的信息等等给到模型，然后给到 Agent，它以一种自己使用上来比较舒服的方式，也就是 Skill，保存下来了。”

Skill 的本质：知识的形式化封装

Skill 不是简单的 prompt 模板，而是将**人类在特定领域的经验、方法和判断逻辑**封装为 Agent 可理解和执行的形式化模块。它的价值在于：

- 1. **跨用户复用**：一个人的最佳实践可以通过 Skill 迅速传播给所有使用该 Agent 的用户。
- 2. **跨模型兼容**：Skill 作为与模型无关的抽象层，可以在不同底层模型之间迁移。
- 3. **渐进式积累**：社区可以像搭积木一样，通过不断创建和分享 Skill 来扩展 Agent 的能力边界。

3.2.2 Skill + COI vs. MCP：两种扩展范式的对比

阿岛在讨论中明确对比了两种让 Agent 获得新能力的范式：

表 1: Skill + COI 范式与 MCP 范式的对比

维度 & Skill + COI & MCP
编写门槛 & 普通用户可写 & 需要工程师编写
传播方式 & 社区分享、即装即用 & 协议对接、集成开发
适用场景 & 个人 workflows、轻量自动化 & 企业系统对接、复杂 API 调用
扩展粒度 & 任务级能力封装 & 工具级接口标准化
用户群体 & 非技术用户友好 & 开发者友好

核心判断：Skill + COI 是 Agent 民主化的关键

阿岛指出：“MCP 是一个当然扩展性好，但他需要人来写，需要工程师来写。但 Skill 加 COI 这种范式，其实普通人都能写。”

这一判断的深层含义是：**Agent 的普及程度取决于扩展它的门槛高低**。如果只有工程师才能为 Agent 增加新能力，那么 Agent 本质上仍是技术人员的工具；而如果普通用户都能通过自然语言描述来创建 Skill，Agent 才能真正成为每个人的“数字同事”。

泽音进一步补充了 Skill 的分享价值：

“一个人写、并且分享，不仅让我的 Agent 能更聪明，而且我能分享给你，让你的 Agent 也变得更聪明。”

这种 **Skill 的社交网络效应**是 Harness 生态扩展的核心驱动力。阿岛甚至将其与 GitHub 类比：“如果 OpenClaw 离开了 Claw Hub，他也不可能那么火。因为大家的 scale 不会那么快地传播开去。”

3.2.3 蒸馏的双向性：谁在蒸馏谁？

Tommy 在讨论中提出了一个引人深思的反向视角：

“我觉得我就是在被 AI 蒸馏……我觉得今天模型训练，包括 Anthropic 在内做的事情，就是在蒸馏人类。”

他指出，Anthropic 和 OpenAI 雇佣了专门的数据公司（如设计公司 AI），让各行各业的人提出“当前 AI 无法回答的问题”并教会它——这个过程本质上就是**人类向模型蒸馏知识**。而模型公司通过这种方式，不断将人类的能力内化到模型参数中。

常见误解：蒸馏是单向的

实际上，蒸馏是双向的过程：

- **人 → Agent**：用户将自己的工作方式、判断逻辑、最佳实践封装为 Skill，让 Agent 继承。
- **Agent → 人**：Agent 的执行轨迹、成功经验成为模型训练数据，反过来提升底层模型的能力。
- **模型 → 全社会**：当模型内化了足够多的 Skill 和工作流后，它实际上吸收了人类社会的集体智慧。

理解这种双向性，对于设计 Harness 的反馈闭环至关重要。

3.3 In-Live Thinking：Agent 与 Chatbot 的本质分野

3.3.1 Chatbot 的结构性局限

在 01:10:00 左右，泽音从技术层面系统阐述了 Chatbot 与 Agent 的根本区别：

“Chatbot 的核心是当下就给你一个回答。虽然你加了 Reasoning，但它其实是不能去做很多的探索，它不能跟这个环境去做非常多的互动。”

这一区别的技术含义极为深远。Chatbot 的架构决定了它的目标是生成一个尽可能合理的文本回复，即使加入了 Chain-of-Thought 或 Reasoning 机制，它的”思考”仍然局限于文本空间内部，不能与外部世界发生实质性的交互。而 Agent 的目标是在真实环境中完成一项任务，这要求它能够：

- **感知环境状态**：读取文件、查询数据库、调用 API、浏览网页。
- **执行动作**：修改代码、发送邮件、部署服务、操作界面。
- **根据反馈调整策略**：当环境反馈与预期不符时，重新评估并选择新的行动路径。

3.3.2 BrowserCamp：Agent 能力的试金石

泽音以 OpenAI 设计的 BrowserCamp benchmark 为例，说明了 Agent 所需的独特能力：

”你要在互联网里面去找非常非常多的 cross 的信息结合在一起，才能找到那个点。你很容易找错——你可以找到满足两个或者三个的，但要找到满足这四个条件的，是一个非常非常难的事情。”

BrowserCamp 要求 Agent 在广阔的互联网信息空间中进行深度探索，有时需要向前深入挖掘，有时发现方向错误需要折返重来。这种探索-验证-修正的循环，正是 Agent 与 Chatbot 的本质差异所在。

BrowserCamp 的技术启示

BrowserCamp 作为 Agentic benchmark 的设计思路值得深入理解：

1. **多条件交叉验证**：正确答案需要同时满足多个独立条件，每个条件分布在不同的信息源中。
2. **高分支因子**：每一步探索都有大量可能的选项，Agent 需要具备有效的剪枝和排序能力。
3. **错误可恢复**：探索过程中不可避免地会走入死胡同，关键在于能否及时发现并回溯。
4. **长程依赖**：最终答案的验证可能需要回溯到最初的几步决策，要求 Agent 维持完整的搜索树记忆。

这种 benchmark 的设计哲学反映了 Agent 能力的核心：不是”一次性答对”，而是”在不确定环境中持续探索直至找到正确解”。

3.3.3 In-Live Thinking：边做边想的实时决策机制

面对上述挑战，泽音引出了 MiniMax 在 M2 模型上重点投入的能力——**In-Live Thinking**：

”In-Live Thinking 就是说你在行动一步一步之后，你要重新思考推演我下一步应该怎么做。”

In-Live Thinking 的技术定义

In-Live Thinking（实时思考）是由 Anthropic 在 Claude Sonnet 4 发布时提出的概念。它的核心机制是：

1. **初始规划**：Agent 在面对任务时首先生成一个初步的执行计划。
2. **执行-观察**：Agent 按计划执行第一步动作，并观察环境的实际反馈。
3. **重新思考**：基于观察到的结果，Agent **重新评估**当前状态与目标的差距，必要时调整后续计划。
4. **迭代循环**：重复执行-观察-重新思考的循环，直至任务完成。

这与“一次性规划后机械执行”的模式形成鲜明对比。泽音用一个生动的比喻说明了其必要性：“你真正运行下去的时候，你会发现这好像跟我想的不一样。我打开以后，我认为应该是一个水阀门，结果它是一个煤气阀门——那我可能要换别的方法。”

常见误解：In-Live Thinking

很多人误以为 In-Live Thinking 就是让模型生成更长的 Chain-of-Thought。但关键区别在于：**In-Live Thinking 的每次重新思考都基于真实的环境反馈**，而非纯文本空间内的自我推演。没有环境交互的“长思考”只是更复杂的文本生成；有了环境交互的“实时思考”才是真正的 Agentic 行为。

泽音进一步指出，这是 **Agentic 和 Chat 最根本的差别**：“就是他真的走入真实世界，真的解决问题，所以他需要不断的思考，不断的行动。”

3.4 模型与应用的共生螺旋

3.4.1 “你永远会被模型内化掉”

在 01:13:00 左右，泽音提出了一个对 Agent 创业者而言略显“悲观”但极具洞察力的判断：

“这可能也是今天做 Agent 的应用，如果是通用 Agent 的应用，比较悲哀的一件事情——就是你永远会被模型内化掉。”

他用 Skill 举例：“你写的 Skill 其实也是或者过去搭的 workflow，其实都是帮 Agent 去完成任务。他完成任务的轨迹也会成为模型去做后续的训练素材和数据。”

共生螺旋的三阶段

模型与应用之间的关系不是简单的“谁取代谁”，而是一个动态的**共生螺旋**：

第一阶段：应用探索 (Unlock)。当新模型发布后，无数应用开发者在真实场景中探索它的能力边界，“把这个模型的能力往外推一步之后，他能够去做哪些事情”。

第二阶段：模型吸收 (Internalize)。模型公司观察这些应用解锁的做法，“重新吸收回它的模型中，然后内化到包括它和 Agent 一起的对外提供的能力里面”。

第三阶段：下一轮释放 (Release)。内化后的模型能力更强，释放出新的可能性，引发下一轮应用创新——循环往复。

3.4.2 Human in the Loop: 人类在循环中的不可替代性

尽管模型在不断地内化应用能力，泽音最后强调了一个重要的底线：

”所以我觉得我们都是 Human in the Loop。就无论你做不做，我觉得我们都是 Human in the Loop。”

这句话有两层技术含义：

1. **方向性的不可替代。**在模型能力扩散的过程中，人类始终负责**定义目标和判断品味**。Agent 可以优化”如何做”，但”做什么”和”为什么做”仍需人类决定。
2. **反馈闭环的必需节点。**即使 Agent 能够自主运行很长时间，最终仍需要人类对结果进行质量评估和价值判断——这个反馈是模型持续改进的必要输入。

阿岛也呼应了这一观点：模型训练过程中，人类工程师负责”基于汇总结果进行方向性判断和品味选择”，而非执行具体的实验步骤。

3.5 本章小结

本章围绕 Agent Harness 的技术内核展开了四个层面的深入讨论：

第一，关系层面。用户与 Agent 从”工具使用者”到”协作者”的心理转变，为 Harness 的个性化、记忆化和异步进化能力提供了需求基础。Self-Evolution 不仅是技术机制，更是对这种新型关系的回应。

第二，扩展范式层面。Skill + COI 范式通过降低能力扩展的门槛，实现了 Agent 的**民主化**。与 MCP 的专业开发者导向不同，Skill 让普通用户都能为 Agent 注入新能力，并通过社区分享形成网络效应。

第三，能力边界层面。In-Live Thinking 是 Agent 区别于 Chatbot 的**结构性特征**——不是更长的推理链，而是基于真实环境反馈的动态策略调整。BrowserCamp 等 benchmark 的设计哲学正是为了衡量这种探索-验证-修正的循环能力。

第四，生态动态层面。模型与应用之间存在一个**共生螺旋**：应用解锁模型能力，模型内化应用经验，释放新的可能性引发下一轮创新。在这个螺旋中，人类始终是”方向定义者”和”品味判断者”，而非被替代的执行者。

核心 Takeaway

1. **Harness 的设计必须服务于”伙伴关系”而非”工具关系”。**记忆沉淀、个性化适配和异步进化是这种关系的技术表达。
2. **Skill 系统是 Harness 生态扩展的核心基础设施。**降低扩展门槛、促进社区分享、形成网络效应——这三者决定了 Harness 能否突破单一团队的能力上限。
3. **In-Live Thinking 是 Agentic 能力的分水岭。**没有环境交互的”长思考”只是复杂文本生成；有了环境反馈的”实时思考”才是真正的 Agent。
4. **模型内化的不是应用，而是应用背后的经验。**通用 Agent 的创业者需要找到自己的”不可内化”壁垒——通常是垂直领域的深度专业知识、独特数据源或特殊信任关系。

4 模型公司与 Agent 的共生关系——竞争、互补与未来形态

本章将视角从纯技术讨论拉升到产业格局层面，探讨一个愈发尖锐的问题：当模型厂商（如 MiniMax）自己也下场做 Agent 时，它们与独立 Agent 开发者之间究竟是什么关系？是竞争对手、合作伙伴，还是某种更复杂的共生体？嘉宾们围绕中国模型与海外领先者的差距、垂直 Agent 的创业机会、Agent 基础设施（infra）的分层逻辑，以及近期行业热点事件（GPT-4.7、Claude Code 强制实名、源代码泄露）展开了激烈而深入的辩论。

4.1 模型公司的视角：为什么 MiniMax 也在做 Agent？

要理解模型公司与 Agent 开发者的关系，首先要回答一个看似矛盾的问题：既然 MiniMax 的核心业务是训练底层模型，为什么它还要投入资源做面向终端用户的 Agent 产品？

MiniMax 的业务布局

MiniMax 是中国领先的通用人工智能公司之一，以自研大语言模型为核心，同时布局了面向消费者的 Agent 产品矩阵。在本场对谈中，嘉宾阿岛（MiniMax 首席架构师）和泽音（MiniMax 研发工程师）多次从“模型公司做 Agent”的内部视角出发，提供了大量一手的行业判断。

4.1.1 一切皆为代码：Agent 的本质洞察

阿岛提出了一个极具穿透力的观点：**Everything is coding**（一切最终都是代码）。在对谈中，他详细阐述了这一洞察的形成过程：

“我们今天也在做通用的办公领域，比如 finance、人事、法律等等。但我们最终发现，everything is coding——最终你去解决它，都是通过某一个 code 的方案去解决它。”

这一观点的深层含义在于，过去我们认为白领工作与“编程”无关，但实际上，每一个 Excel 公式、每一个 PPT 动画、每一个 Word 邮件合并，本质上都是在创造**小型软件**。阿岛进一步指出：

“Office 三件套并不是最原生的数据格式。DOCX、XLSX、PPTX 都是在原始数据基础上套了一层软件——包括界面和逻辑。所以每次我发一个 Excel 给你，我发的不是一堆原始数据，而是数据加上逻辑和界面——因此我发你的其实是一个软件。”

核心概念：代码作为通用接口

如果一切白领工作的本质都是创造小型软件，那么 **coding 能力就是模型触达真实世界的最通用接口**。这也是为什么 Anthropic 押注 coding 方向后发展势头迅猛——coding 是最能 touch 到真实世界边界的领域，而不是解一个只有数学家才能解的数学问题（那只是真实世界分布中非常小的一个逆起方向）。

从这个视角出发，MiniMax 做 Agent 并非“不务正业”，而是用 **Agent 作为探针，去探测模型能力的边界**，并反过来用这些真实世界的任务反馈来蒸馏、优化底层模型。这是一个双向增强的闭环。

4.1.2 中国模型与海外领先者的真实差距

在讨论模型公司为什么做 Agent 时，不可避免地要直面一个敏感话题：中国模型与 OpenAI、Anthropic 的差距到底在哪里？阿岛给出了非常坦诚且结构化的分析：

差距不大的方面：训练方法和对模型训练的认知。阿岛提到，MiniMax 与硅谷的研究员有持续交流，”坦白说，差距没有那么大”。

真实差距的两个方面：

1. **任务定义的科学性**。OpenAI 和 Anthropic 会请各个领域最优秀的人——学术领域最优秀的博士、工业界最顶尖的专家——来一起”劝模型”（即定义模型需要解决的任务）。他们有一套非常科学的方法：先定义任务，再蒸馏这些最优秀的人成为训练数据，再与最优秀的企业合作构建 Agent 去解决问题，进入”能力不断扩散、不断接触更复杂真实世界任务”的 loop。
2. **对 AGI 路径的远见与算力支撑**。Anthropic 比中国公司更早看到 AGI 的方向，并在 OpenAI 续完 GPT-4.5 后放弃更大规模模型时，**坚定地压住 coding 方向**，最终获得了回报。这背后确实有算力差距——”如果我们能有那么多的算力，相信中国有足够多的人才，也能支撑我们去做足够多的实验，找到那条 scale 的路径。”

常见误解：差距已被消除？

泽音补充了一个重要的纠偏视角：存在两种模糊的认知。第一种是”大家都在用 AI 了，国产模型也能用，差距基本消除”。但从 GDP 生产总值的角度，只有当模型真正有能力在所有生产场景、多模态场景上发挥功效时，我们才能”更加有底气地说出这句话”。衡量标准应该是：**模型生产的价值占整个社会 GDP 的百分之多少**——这是 Dario (Anthropic CEO) 在 2025 年 1 月明确提出的评估标准。

4.2 COI 困境：模型厂商做 Agent 的利益冲突

当模型公司既做底层模型、又做上层 Agent 时，一个天然的**利益冲突 (Conflict of Interest, COI)** 就出现了。本节围绕 Agent 基础设施 (infra) 的分层逻辑，深入剖析了这一困境。

核心概念：COI (Conflict of Interest)

COI，即利益冲突，指模型厂商同时扮演”基础设施提供者”和”应用层竞争者”双重角色时产生的结构性矛盾。类比来说，这就像一家电力公司同时经营着耗电最大的工厂——它是否有动力让其他工厂也能低成本获得同样优质的电力？

4.2.1 Agent Infra 的两层结构

阿岛将 Agent 基础设施分为两个层次，这一分层框架对于理解创业机会至关重要：

第一层：社会基础设施层——身份认证 (identity) 和支付 (payment)。

”这两个是最核心的问题。你想想移动互联网最终解决这个问题的人是谁？是微信和支付宝——这是 PC 互联网就存在的两个公司。因为它会变成整个社会的基础设施。”

阿岛明确指出，这一层**不太是创业公司能够承担的**。无论是责任、需要撬动的资源，还是说服所有人的难度，都超出了创业公司的能力边界。

第二层：业务应用层——面向 Agent 的工具和环境。

例如，一个能让 Agent 方便挂号的 COI (这里 COI 指医疗系统的接口，注意与利益冲突 COI 区分)，一个在身份问题解决后能让 Agent 方便缴费或打车的接口。阿岛认为：

”这一层是比较业务和应用层的，我觉得是有机会的。但这一层的机会取决于你在那个领域的 know-how——你得在那个领域里面有一些垂直的经验，或者已经有的一些数据库。”

4.2.2 垂直 Agent 的创业路径：两阶段演化

阿岛进一步提出了垂直 Agent 领域的两阶段演化模型：

第一阶段：传统玩家的 AI 化。原有的领域玩家（如滴滴、美团）尝试融入 Agent 能力，提供面向 Agent 的接口。

第二阶段：AI 原生产品的涌现。当 Agent 开始接入这些环境、具备这些能力之后，”所有东西围绕着 AI 去构建”，就会出现新的产品范式的创新机会。

常见误解：Agent Infra 创业公司机会遍地？

对于 sandbox（沙箱）、memory infra、runtime infra 等方向，阿岛的观点非常明确：“我觉得没有那么本质。”这些更偏工具层的方向，最终可能被云厂商或模型公司自身吸收，难以形成独立的、有护城河的创业机会。

4.3 Agent 会取代模型吗？还是相辅相成？

在厘清 infra 分层和 COI 问题后，讨论自然推进到一个更根本的问题：Agent 和模型之间究竟是替代关系还是互补关系？通用 Agent 与垂直 Agent 谁将主导未来？

4.3.1 通用 Agent vs 垂直 Agent：嘉宾观点碰撞

阿岛的观点：垂直 Agent 解决最后一公里，但通用 Agent 终将吞噬一切。

阿岛认为，垂直 Agent 今年会”百花齐放”，因为它们解决的是最后一公里交付的问题：“通用就注定了你不可能去那么的定制化。”垂直 Agent 的价值在于，当通用 Agent 和模型已经有能力、但”就差那一步”才能在整个行业里跑起来的时候，垂直 Agent 补上了这关键的一环。

然而，阿岛同时做出了一个更激进的判断：

”今天大家已经看到 AI 在逐渐替代 SaaS 了——这其实就是某种垂直的 Agent。但如果

没有壁垒的东西，我觉得都会被通用 Agent 吃掉。”

泽音的反驳：模型能力边界决定 Agent 形态。

泽音对”通用 Agent 终将吞噬一切”的论断持保留态度。他认为，很多垂直领域的表现差异，**本质上是底层模型能力不够强**，而不是通用 Agent 的范式有问题：

”我觉得这只是视频理解模型不够强，而视觉视频理解和生成模型正在飞速进步。我觉得最终通用 Agent 它也能做——那只是一个交互的问题。”

4.3.2 法律：通用 Agent 走不通的”最后一公里”？

阿岛举了一个他认为通用 Agent 难以彻底攻克的领域——**法律**：

”法律是一个很严肃的问题。你要正式出一份给客户的意见，要考虑合规的成本和合规的风险。他没有标准答案——模型可以给到你几个建议，但最终要出到一份意见书，跟客户讨论说我们在海外要遵循某个国家的合规，我最后就采取某个路径，然后我就承担这个责任、承担这个风险。”

这个例子揭示了一个深层规律：当任务的最终输出涉及法律责任、需要人类明确承担后果时，Agent 很难完全替代人类的”最后一公里”决策。这不是技术问题，而是社会制度问题。

核心概念：模型与 Agent 的边界

模型提供通用认知能力，Agent 提供任务编排与环境交互能力。二者的边界并非固定不变——随着模型能力增强，原本需要复杂 Agent 架构才能完成的任务，可能简化为几次模型调用。但责任归属、合规要求、领域 know-how 构成的”社会边界”，短期内不会消失。

4.4 未来产品形态畅想：从 IDE 到自然交互

本节借近期行业热点事件（GPT-4.7 发布、Claude Code 强制实名、Anthropic 源代码泄露、Manager Agent 架构），展望 Agent 产品形态的未来演化方向。

4.4.1 GPT-4.7：RL 路线的胜利与代价

阿岛从模型训练视角对 GPT-4.7 做了精准的技术拆解：

”4.5 是一个强 SFT 的模型，RL 没有做那么多。但从 4.6 你就能比较明显看到它在加强 RL 了。4.7 我觉得是一个非常强 RL 的模型——包括它那个 x-high 档位就太顶了，就 GPT 也太顶了。”

然而，强 RL 也带来了典型的副作用：

”RL 就逃不脱啊——因为他是最终 reward，他对过程、包括幻觉这样的问题，其实是没有那么多的控制或者奖励/惩罚的。所以你看到很多人吐槽 4.7 的约束、真人实验还没有 4.6 好——这是 RL 非常典型的一个问题，因为他只管最终结果。”

阿岛认同 Hugging Face CEO 的观点：OpenAI ”征留”了那个 milestone——这是一个战略性的产品节奏决策，”换我是 OpenAI，我也会这么做”。

4.4.2 Claude Code 强制实名：安全还是垄断？

Claude Code 强制用户实名认证的事件，引发了嘉宾之间最激烈的观点碰撞。

泽音：善意假设——AI 时代的身份归因难题。

泽音从 Agent 基础设施的角度给出了一个技术性的解读：

”在 AI 时代，认证一个 Agent 来自哪里、归属于谁，或者 Agent 发出请求很容易，但他想自证自己是谁，其实反而是一件难的事情。”

他认为，当 AI 已经能够做很多事情时，**行为该如何归因**成为一个根本问题。强制实名虽然 imperfect（”我也可以说不小心扫了人脸，别人在用我的电脑”），但暴露了一个巨大的机会：**如何证明一个来自 AI 的请求真正来自哪里。**

阿岛：批判视角——安全叙事背后的垄断野心。

阿岛对泽音的”善意假设”直接回应：”我觉得泽音是比较善意假设，我就没有那么善意了。”

他拆解了 Anthropic 的逻辑链条：AI 很强大 → 要约束它避免对人类产生危害 → 要确保 AI 不被”邪恶的国家”和”邪恶的人类”使用 → 所以要 safety → 所以要实名。

”他当然在他的逻辑下是自洽的。但凭什么你来定义这件事情？这至少说明我对民主的认知是不太一样的。至少我们的观点叫 intelligence with everyone，而不是你扫了人脸的才可以。”

常见误解：强制实名 purely 出于安全考虑

阿岛承认”他肯定是有安全的考虑，这点我是认可的”，但强调”我觉得不只有这一点”。泽音则从算力约束角度提供了另一个解读：Anthropic 可能面临”发了极为优秀的模型但又不能让大家用”的尴尬——如果模型真像宣称的那么厉害，需要投入的推理算力将是天文数字。”如此强大的模型在这个时间点推出去，可能能够把口碑造起来，但如果不让人家用，那又有什么意义呢？”

4.4.3 Manager Agent 与源代码泄露：封闭 vs 开放的博弈

泽音结合 Anthropic 近期发布的 Manager Agent 架构，提出了一个更深层的担忧：

”他把人的脑和手完全分开了。一切 AI 的思考以及真正想出一个点子来、要去执行的动作都在云上进行。用户只能通过一个授信给 AI 的环境看到他去做某些动作。”

这是否意味着 Anthropic 不想让自己模型的深层次思考暴露给用户？”他想做一个真正的掌控一切的人——只让我的模型就像是一个远程的桌面助理一样，开始操作你的电脑。”

而近期 Claude Code 源代码泄露事件，则意外地给了行业一个”祛魅”的机会。泽音作为 Agent 研发工程师，第一反应是”兴奋”——”发现有一个最佳实践可以去学习了”。更重要的是：

”看到源代码之后，发现其实也没有那么多的魔法在里面。他也是有大量的假设、探索和摸索。特别是和 Codex 比较——Codex 是一个极度简化的框架，把一切都交给模型了；但 Claude Code 就像所有人说的，Anthropic 自己也不相信自己的模型，他要在一切地方都给他约束好。”

核心概念：开源 vs 封闭的动态平衡

源代码泄露事件揭示了一个行业真相：**即使拥有无限算力和最好智能的公司，在 Agent 和通用 Agent 方面也是处在探索和实践的阶段。**这为所有参与者——包括中国的模型公司和 Agent 开发者——提供了赶超的窗口期。”只要你有能力去验证自己的想法，并且尽量快速地把它落地，你就是有机会赶超他们的。”

4.5 本章小结

本章从产业格局视角审视了模型公司与 Agent 开发者的复杂关系，可以提炼为以下核心要点：

1. **模型公司做 Agent 的必然性。**Agent 是模型触达真实世界的探针，coding 是一切白领工作的通用接口。MiniMax 做 Agent 不是为了与应用层竞争，而是为了在真实任务中蒸馏、优化底层模型。
2. **COI 是结构性矛盾，但 infra 分层创造了合作空间。**身份认证和支付等社会基础设施层由巨头承担，而业务应用层（垂直领域的 know-how 和数据积累）仍有创业机会。垂直 Agent 将经历”传统玩家 AI 化 → AI 原生产品涌现”的两阶段演化。

3. **通用 Agent 与垂直 Agent 不是零和博弈。**通用 Agent 的能力边界不断扩张，会吞噬大量原本需要垂直方案的领域；但法律责任、合规要求、领域 know-how 构成的”社会边界”短期内不会消失。二者是动态演进而非静态替代的关系。
4. **行业热点背后的深层逻辑。**GPT-4.7 代表了 RL 路线的胜利与代价；Claude Code 强制实名折射出安全叙事与垄断野心的交织；源代码泄露则证明 Agent 领域尚无不可逾越的壁垒——**快速验证、快速落地，仍是这个时代最有效的竞争策略。**

延伸思考

- 如果”everything is coding”成立，那么未来的”白领”是否都会变成某种意义上的”软件工程师”？
- 当模型厂商同时做基础设施和应用层时，反垄断监管应该如何介入？
- 在 Agent 行为归因尚未解决的时代，”intelligence with everyone”的愿景与”安全可控”的需求之间，是否存在不可调和的张力？

5 开源、安全与人类未来——行业争议与乐观主义收尾

在前面的章节中，我们深入探讨了 Agent Harness 的技术架构、工程实践与产品形态。然而，任何一场关于 AI 的深度对话，最终都不可避免地要回到那些更具根本性的问题上：技术应该开放还是封闭？AI 会不会取代人类？面对飞速发展的技术，我们应该恐惧还是拥抱？本章将梳理三位嘉宾对这些行业关键争议的独特见解，提炼出超越技术层面的深层思考，并以积极的态度为整场对话收尾。

5.1 开源 vs 封闭：两条路径的博弈

对话进入后半段，嘉宾们围绕行业生态的开放性问题展开了讨论。一个核心观察是：即便是世界上最聪明、拥有最强模型的公司，其思考路径与创业者们也在同一个“大气层”内。

技术趋同的必然性

嘉宾提到，在查看某头部模型公司的开源代码时，发现其中包含大量 AB 测试和尚未灰度发布的功能，“其实和我们在想的是类似的”。这种趋同并非偶然——当技术发展到一定阶段，行业对“什么是对的”会逐渐形成共识。

嘉宾们以 Cursor、Claude Code、Codex 等产品为例，指出一个有趣的现象：“把 logo 蒙上好像都差不多”。这引出了两种可能的解读：

- **短期共识论**：当前模型能力决定了交互范式的上限，因此优秀产品的形态必然趋同；
- **生命周期论**：每一代产品都有其生命周期，随着模型进步，交互范式会不断迭代，“一代版本一代神”。

嘉宾更倾向于后者。他们认为，从 Copilot 到 Cursor，再到 Claude Code，乃至今天用 OpenAI 的模型指挥多个 Claude Code Agent，行业正在经历一个“外层越来越薄”的过程——产品形态在不断向更高层级抽象，但底层逻辑始终围绕“让模型更好地服务人类”这一核心目标。

核心观点：产品形态的生命周期

嘉宾指出：“agent 这一层是有生命周期的，它就是不断地随着模型进步在更新换代。”去年 Manus 定义了“展示操作过程和结果”的范式，而今年模型能力更强后，行业共识转向了“只给结果，不展示过程”——因为用户已经足够信任模型，不再需要“看模型干活”的爽感。

5.2 AI 安全与对齐：被忽视的问题

在讨论产品形态的同时，嘉宾们也触及了 AI 安全与对齐这一更为深层的话题。

安全问题的紧迫性

尽管本场对话的主要焦点是 Agent Harness 的工程实现，但嘉宾们多次暗示：当 AI 系统获得越来越强的行动能力（harness）时，安全与对齐问题将变得前所未有的重要。一个能够自主操作浏览器、编写代码、调用 API 的 agent，其潜在风险远高于一个只能生成文本的聊天机器人。

嘉宾们提到，当前行业对“模型是智能，harness 是手脚”的共识，实际上也隐含了一个安全命题：如果“手脚”（工具调用能力）不受约束，再聪明的“大脑”也可能造成不可控的后果。这要求开发者在设计 harness 时，必须将权限控制、沙箱隔离、人类监督等安全机制作为一等公民来考虑，而非事后补丁。

5.3 “AI 替代人类”：嘉宾们的不同观点

对话进入最后阶段，主持人提出了一个尖锐的问题：“关于 agent，有没有什么观点是今天大多数人认可，但你们不认可的？”

嘉宾们不约而同地将矛头指向了“AI 将取代人类”这一流行叙事。

核心观点：AI 是水、电、蒸汽机

第一位嘉宾旗帜鲜明地表达了反对立场：

“我觉得人类的创造力是无可替代的。我仍然觉得 AI 是水，是电，是蒸汽机，但最终驾驭它和创造出那些美好东西的是人类。”

这一观点得到了另外两位嘉宾的呼应与补充，形成了三个层次的论证：

第一层：历史类比——ATM 机的启示

ATM 机与银行从业者的历史

第二位嘉宾以 ATM 机的出现为例，提供了一个极具说服力的历史类比：

“上个世纪，银行是一个体面的工作。ATM 机出现后，大家慌了——银行是不是要裁员了？但实际情况是：ATM 机提高了效率，银行没有变少，而是变多了；在银行里工作的人也没有变少，而是变得更加多了。”

这一案例揭示了一个被忽视的真相：技术提效往往会扩大整个市场的规模，创造出更多新的岗位和需求，而非简单地消灭旧岗位。

第二层：数据支撑——人类社会的长期向好

《人性中的善良天使》与历史曲线

嘉宾引用了两本著作来支撑其乐观主义立场：

- 《人性中的善良天使》（史蒂芬·平克）：用大量数据证明，尽管人类总是倾向于悲观，但人类社会实际上在变得越来越好——战乱、犯罪率等指标长期下降。
- 《原则》第三部（瑞·达利欧）：书中展示了人类 GDP 的指数级增长曲线，即便两次世界大战在曲线上也只是“微小的停顿”。

第三层：个人行动——打不过就加入

核心观点：拥抱新版本的”蒸汽机”

第三位嘉宾以更为务实的态度回应了替代焦虑：

”虽然我相信人不会被替代，但我当初加入这个行业的理由是：当我用 ChatGPT 帮我写代码之后，我意识到——打不过就加入。相当于你有了蒸汽机、有了电力之后你不用，就相当于人家用热兵器，你用冷兵器，那你肯定要被怼死。”

5.4 乐观主义收尾：拥抱新版本的”蒸汽机”

对谈在三位嘉宾的乐观主义共识中走向尾声。他们共同构建了一个关于 AI 时代的积极叙事框架：

1. **技术是中性的工具**：AI 不是洪水猛兽，而是像水、电、蒸汽机一样的基础设施；
2. **人类创造力不可替代**：驾驭工具、创造美好事物的始终是人类；
3. **历史站在乐观者一边**：从蒸汽机到电力，从 ATM 到互联网，每一次技术革命都扩大了人类的可能性边界；
4. **行动胜于焦虑**：与其担心被替代，不如尽早拥抱 AI，将其作为伙伴、提效工具或长期挚友。

警惕两种极端

嘉宾们的乐观并非盲目。他们同时提醒听众警惕两种极端心态：

- **盲目悲观**：认为 AI 将消灭所有工作，从而陷入焦虑与停滞；
- **盲目乐观**：认为 AI 会解决一切问题，从而忽视安全、对齐与伦理问题。

真正的智慧在于：既相信人类社会的长期向好，又积极行动、主动适应技术变革。

金句收尾：新版本来了

对谈的最后，嘉宾以一句话为整场直播画上了句号：

”新版本来了，还是要看一下新版本的玩法。相信是新版本，不是滔天大浪把我淹没掉，而是他会成为让我们可以在里面自由翱翔的一种舞台。”

5.5 本章小结

本章围绕 AI 行业的三个核心争议展开讨论：

1. **开源与封闭**：嘉宾们认为产品形态的趋同是技术发展的阶段性现象，随着模型进步，交互范式将持续迭代，”一代版本一代神”。
2. **AI 安全**：当 agent 获得越来越强的行动能力时，安全与对齐必须从设计之初就作为核心考量，而非事后补丁。
3. **替代焦虑**：通过 ATM 机的历史类比、《人性中的善良天使》的数据支撑，以及”打不过就加入”的务实态度，嘉宾们共同构建了一个乐观而理性的叙事——AI 是蒸汽机的新版本，驾驭它的始终是人类自己。

从 Agent Harness 的技术架构，到工程实践中的血泪教训，再到产品形态的演进趋势，最后落脚于对人类未来的乐观展望——这场深度对话为我们勾勒了一幅完整的 AI 时代图景。技术在不断迭代，但有一些东西始终不变：人类对创造的热爱、对美好的追求，以及面对变革时的勇气与智慧。

6 总结与延伸

6.1 对谈的核心脉络回顾

这场由 MiniMax 首席架构师阿岛、研发工程师泽音与 Hermes Agent 产品策略负责人 Tommy 共同参与的深度对谈，围绕“Agent Harness”这一核心概念，从五个维度构建了一幅完整的认知图景：

1. **行业背景**（第 1 章）：2025 年春节后，从 OpenClaw（“养虾”）到 Hermes Agent（“驯马”），中国用户对 Agent 的认知完成了从“听说过”到“天天用”的跃迁。用户心理从“工具使用者”扭转为“协作者”，这种关系转变为 Harness 的设计提供了需求基础。
2. **开源 Harness**（第 2 章）：Hermes Agent 不是另一个 Chatbot，而是一个开放的 Agent 基础设施。其 Self-Evolution 机制让 Agent 能从真实反馈中沉淀 Skill，Multi-Agent 架构则系统性解决了上下文衰减和纠错能力的结构性瓶颈。
3. **技术内核**（第 3 章）：Skill + COI 范式通过降低扩展门槛实现了 Agent 的民主化；In-Live Thinking 基于真实环境反馈的动态策略调整，是 Agent 区别于 Chatbot 的结构性特征；模型与应用的共生螺旋则揭示了“你永远会被模型内化”的行业规律。
4. **产业格局**（第 4 章）：模型公司与 Agent 开发者之间不是零和竞争，而是共生关系。模型需要 Agent 作为“探针”探测能力边界，Agent 需要模型提供智能引擎。“Everything is coding”的洞察揭示了 Agent 作为通用接口的深层价值。
5. **人类未来**（第 5 章）：开源与封闭的博弈、AI 安全与对齐、“AI 替代人类”的焦虑——这些争议在技术快速迭代的背景下愈发尖锐。但嘉宾们以 ATM 机的历史类比和“蒸汽机新版本”的乐观叙事，为我们提供了理性而积极的思考框架。

6.2 我们的结构化蒸馏

作为这场对谈的学习者，我们从技术、产品和哲学三个层面提炼出以下核心认知：

技术层面

- **Harness 是模型的放大器，不是替代者。**它的核心价值在于约束、编排和驱动——让模型的智能在复杂 workflows 中发挥最大价值。
- **In-Live Thinking 是 Agentic 能力的分水岭。**没有环境交互的“长思考”只是复杂文本生成；有了环境反馈的“实时思考”才是真正的 Agent。
- **Skill 系统是生态扩展的基础设施。**降低门槛、促进分享、形成网络效应——这三者决定了 Harness 能否突破单一团队的能力上限。

产品层面

- **Agent 的设计必须服务于”伙伴关系”。**记忆沉淀、个性化适配和异步进化是”伙伴”关系的技术表达，而非旧式”工具调用”的延伸。
- **通用 Agent 的创业者需要找到自己的”不可内化”壁垒。**垂直领域的深度专业知识、独特数据源或特殊信任关系，是抵抗”被模型内化”的三种可能路径。
- **开源与封闭不是对立，而是互补。**开源 Harness 提供基础设施和生态，封闭产品提供极致体验和垂直优化——两者共同推动行业向前。

哲学层面

- **蒸馏是双向的。**人类向 Agent 封装经验，Agent 向模型反馈轨迹，模型向全社会扩散能力——这是一个相互增强的循环。
- **Human in the Loop 不是妥协，而是必需。**方向定义、品味判断和价值评估，这些人类独有的能力不会被替代，而是成为整个系统的”方向盘”。
- **AI 是蒸汽机的新版本。**历史上的每一次技术革命都伴随着”替代焦虑”，但最终结果都是人类生活水平的整体提升。关键在于拥抱变化、学习驾驭。

6.3 开放问题与下一步

这场对话留下了一些值得持续关注的开放问题：

1. **Harness 的标准化：**当越来越多的团队开发自己的 Harness 时，是否会出现类似操作系统或浏览器引擎那样的标准化竞争？MCP、COI、Skill 等多种范式能否收敛？
2. **Agent 安全的系统性方案：**随着 Agent 获得越来越强的行动能力，现有的安全框架（如 RLHF、Constitutional AI）是否足以应对？是否需要全新的安全范式？
3. **中国模型的 AGI 路径：**在算力和数据差距客观存在的情况下，中国模型公司能否找到差异化的 AGI 路径？Agent 是否是一个可能的超车弯道？
4. **Skill 经济的商业模式：**当 Skill 成为 Agent 生态的核心资产时，Skill 的创作者如何获得可持续的回报？这会催生怎样的新型经济形态？

新本来了，还是要看一下新版本的玩法。

相信是新版本，不是滔天大浪把我淹没掉，
而是他会成为让我们可以在里面自由翱翔的一种舞台。